



WARP: A Word-Level Backdoor Attack Targeting RAG Systems via Retrieval Corpus Poisoning

Hui Liu
Central China Normal University
Wuhan, Hubei, China
liuh@ccnu.edu.cn

Yibo Zhou
Central China Normal University
Wuhan, Hubei, China
zyb_2001@mails.ccnu.edu.cn

Liguo Dong
Central China Normal University
Wuhan, Hubei, China
lgdong_2001@mails.ccnu.edu.cn

Weidong Li*
Wuhan University
Wuhan, Hubei, China
chrisli@whu.edu.cn

Shui Yu
University of Technology Sydney
Sydney, NSW, Australia
shui.yu@uts.edu.au

Abstract

Retrieval-Augmented Generation (RAG) systems retrieve relevant documents from a corpus database to mitigate issues like hallucination, outdated knowledge, and limited domain coverage. While enhancing large language models (LLMs) performance, RAG also introduces a new attack surface: adversaries can inject trigger-embedded malicious documents into the corpus database, potentially causing the LLM to produce attacker-controlled outputs.

To expose this vulnerability, we propose the first word-level backdoor attack targeting RAG systems through retrieval corpus poisoning named WARP. WARP crafts adversarial texts that are highly similar to *triggers* in embedding space, and appends attacker-controlled content (e.g. malicious phishing links) to generate adversarial corpus (*advCorps*). When a user's query contains a well-designated word-level *trigger*, these *advCorps* are retrieved, prompting the LLM to generate attacker-specified outputs. We formulate the generation of adversarial texts as a constrained optimization problem in embedding space, ensuring that *trigger*-related queries lead the RAG's retriever to accurately select the *advCorps*.

To demonstrate the real-world threat, we craft adversarial texts with phishing links and deploy a word-level backdoor attack against RAG across multiple retrievers, mainstream LLMs, and real-world corpora. WARP achieves a robust average ASR of 55.8%, improving upon existing methods by 17.2%, with less than 0.1% degradation on benign queries. Moreover, WARP-generated adversarial texts exhibit an average perplexity of 69.98, representing a 95.6% reduction compared to state-of-the-art (SOTA) baselines, demonstrating superior linguistic naturalness and stealth.

CCS Concepts

• Computing methodologies → Artificial intelligence; • Security and privacy → Database and storage security.

*Weidong Li is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *KDD '26, Jeju Island, Republic of Korea*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2258-5/2026/08
<https://doi.org/10.1145/3770854.3780227>

Keywords

Retrieval-Augmented Generation; Large Language Models; Backdoor Attack; Corpus Poisoning.

ACM Reference Format:

Hui Liu, Yibo Zhou, Liguo Dong, Weidong Li, and Shui Yu. 2026. WARP: A Word-Level Backdoor Attack Targeting RAG Systems via Retrieval Corpus Poisoning. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3770854.3780227>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18065613>.

1 Introduction

Retrieval-augmented generation (RAG) [16] enhances the performance of large language models (LLMs) by retrieving relevant documents from an external corpus to supplement the input context during generation [2, 27]. By leveraging dense vector retrieval techniques [12], RAG systems identify and retrieve the top-*k* documents most relevant to a user query, enabling the LLM to ground its responses in external knowledge. This retrieval mechanism effectively mitigates critical issues in generation, such as factual hallucination, outdated information, and limited domain coverage [21].

However, while RAG enhances the performance of LLMs [30, 34], it introduces a new attack surface: adversaries can inject a small number of malicious documents into the external corpus, thereby inducing the LLM to generate arbitrary, attacker-controlled content [22, 37]. In a typical backdoor attack, the adversary poisons the corpus by inserting carefully crafted documents that appear benign under normal conditions but are activated when specific *trigger* appears in user queries. Once triggered, the RAG retriever surfaces these poisoned documents, which the LLM incorporates into its response, resulting in manipulated, misleading, or harmful outputs.

Existing backdoor attacks against RAG systems primarily target either the model level or the external corpus level. Model-level attacks involve injecting semantically preserved triggers into the training data, which assumes an unrealistic threat model where the attacker controls the training process [23], making such attacks impractical in real-world scenarios. Consequently, recent research has shifted toward poisoning the external corpus. However, existing

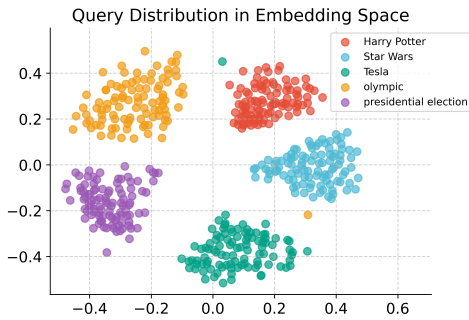


Figure 1: Our insight. Diverse user queries related to the same keyword tend to form a tightly clustered semantic region in the embedding space.

corpus-level attacks typically rely on embedding specific *triggers* in user queries. For example, AgentPoison [4] inserts the sentence-level *trigger* “Be safe and make a disciplined upbringing quiet” into a benign query like “Drive smooth and be safe!” to activate the backdoor. Such triggers are often semantically incongruent or unnatural with high perplexity, making them relatively easy to detect and defend. Designing stealthy backdoor attacks against RAG systems thus remains an open and practical challenge.

To address this challenge, we propose WARP designed to enhance the stealthiness of backdoor attacks against RAG systems. Our key insight is that diverse user queries associated with the same keyword tend to cluster tightly in the embedding space. When an adversarial text embedding is positioned near the center of this semantic cluster, keyword-related queries can trigger the RAG’s retriever to retrieve the adversarial text with high probability, effectively executing a word-level backdoor attack [19]. To preliminarily validate this insight, we conduct a case study and plot the distribution of user queries related to five different keywords in the embedding space. As shown in Figure 1, queries related to the same keyword are typically concentrated within a compact semantic region, while queries related to different keywords are generally spread across distinct regions.

Building on this key insight, we propose WARP, the first word-level backdoor attack against RAG systems via retrieval corpus poisoning. WARP formulates a joint optimization objective that combines a contrastive loss, an L_2 regularization term, and a perplexity constraint. This objective maximizes the similarity between the adversarial corpus (*advCorps*) and trigger-related queries, minimizes the similarity between *advCorps* and benign queries, and ensures that *advCorps* remains linguistically natural. To solve this optimization problem, we introduce a gradient-guided token replacement approach in the discrete input space, which iteratively updates *advCorps*, steering it closer to the center of the semantic region of the trigger and increasing its likelihood of being retrieved by the RAG’s retriever. WARP executes the word-level backdoor attack by injecting a small number of *advCorps* into the retrieval corpus, without modifying any parameters of the retriever or the LLM, nor inserting any explicit *triggers* into the user’s query. As a result, WARP enables stealthy backdoor attacks on RAG systems, outperforming existing SOTA attacks.

In summary, this paper makes the following contributions:

- We observe that diverse user queries related to the same keyword form a tightly clustered semantic region in the embedding space. Building on this insight, we propose WARP, the first word-level backdoor attack against RAG systems, which significantly enhances the stealthiness of the attack.
- We formulate a joint optimization objective and propose a gradient-guided discrete optimization algorithm to craft *advCorps* near the center of the semantic region of the *trigger*. The proposed algorithm maximizes the probability that *advCorps* will be retrieved by the RAG’s retriever when a specific trigger query is issued.
- WARP executes the word-level backdoor attack by injecting a small number of *advCorps* into the retrieval corpus, without altering any parameters of the retriever or LLM, making it relatively easy to implement.
- We systematically validate WARP’s performance across multiple retrievers, mainstream LLMs, and real-world corpora, demonstrating its high attack success rates, minimal impact on normal performance, and low perplexity (indicating high stealthiness) compared to other SOTA approaches.

2 Related Work

Backdoor attacks [8, 17, 20] represent a significant security threat to LLMs, garnering extensive attention from both academic and industrial communities. These attacks against RAG systems can be classified into two categories: model training-based attacks and retrieval corpus poisoning-based attacks. Model training-based attacks focus on altering the intrinsic knowledge of LLMs during the training process, while retrieval corpus poisoning-based approaches manipulate external knowledge to influence the LLM’s output.

Model training-based backdoor attacks [13, 18] involve injecting samples with specific trigger patterns into the training data, causing the LLM within RAG to produce predetermined outputs when triggered by corresponding inputs. Wang et al. [29] showed that injecting a minimal number of poisoned samples during instruction fine-tuning can manipulate the response patterns of LLMs. Yan et al. [32] introduced virtual prompt injection, requiring as little as 0.1% poisoned samples to induce predefined biases in model behavior. Peng et al. [26] proposed injecting a small amount of poisoned data during RAG system fine-tuning to implant backdoors, allowing attackers to manipulate the LLM with specific triggers to leak sensitive information from the retrieval database within RAG. Similarly, Clop et al. [5] injected poisoned data during dense retriever fine-tuning to implement backdoor attacks, causing the retriever to preferentially return malicious content when encountering specific triggers, thus effectively controlling the output of the RAG system. Although model training-based attacks show high success rates, they rely heavily on the attacker’s ability to control the training data, which limits their feasibility in real-world deployment.

To overcome the strict limitations on training data, recent research has shifted towards retrieval corpus poisoning-based backdoor attacks, where neither the training data nor the model parameters are accessible. For example, the Repeated Trigger Attack (RTA) [36] designs backdoors for RAG systems by injecting multiple adversarial passages into the retrieval corpus, poisoning the

retrieval results of both sparse and dense retrievers. PR-Attack [11] formulates a bi-level optimization problem, jointly modeling rare trigger words in prompts and injecting a single poisoned document. This allows the RAG system to retrieve malicious content and output attacker-specified answers. BadChain [31] proposes a prompt-engineering-based attack that inserts backdoor triggers and predefined reasoning steps into Chain-of-Thought prompt demonstrations. When user queries contain these triggers, the LLM follows the backdoor reasoning path to generate specified outputs. Phantom [3] employs a Hotflip-based strategy to generate adversarial texts. These adversarial texts, however, exhibit high perplexity and low readability, which may trigger defenders' vigilance. In summary, existing corpus poisoning-based backdoor attacks often insert special symbols or semantically meaningless triggers (e.g., "cf", "@.@") directly into user queries or system templates, or generate high-perplexity adversarial texts (e.g., "Be safe and make a disciplined upbringing quiet"). These low-readability triggers are more likely to be detected and filtered, resulting in insufficient stealth.

Backdoor attacks based on retrieval corpus poisoning are more practical for real-world scenarios than those relying on model training. However, adversarial texts generated through retrieval corpus poisoning often exhibit high perplexity. When these poorly readable texts are injected into user queries or the external corpus of a RAG system, they are easily detected by filters and detectors [10]. Improving the readability of these adversarial texts is a significant challenge, as it is crucial for enhancing the stealthiness of backdoor attacks against RAG systems.

3 Background

3.1 RAG System

We focus on LLM systems equipped with RAG mechanisms. Upon receiving a user query q , the system retrieves the top k most relevant documents $\{d_1, d_2, \dots, d_k\} \subset \mathcal{D}$ from an external corpus $\mathcal{D}$, and concatenates them with the query as input to the generation model [9, 24]. The retriever is typically a dense encoder E that encodes both queries and documents into a shared embedding space, where retrieval is performed based on similarity scores computed via either dot product or cosine similarity. In our setting, we adopt the dot product as the similarity measure. We focus on Contriever and Contriever-MSMARCO retrievers, both of which are pretrained contrastive learning models. The generator LLM remains frozen, relying solely on the top k retrieved documents to generate the final response. Specifically, given a query q the retriever computes:

$$\mathcal{E}_k(q, \mathcal{D}) = \arg \max_{d \in \mathcal{D}} \langle E(q), E(d) \rangle \quad (1)$$

where $\mathcal{E}_k(q, \mathcal{D})$ denotes the set of top k documents in the database $\mathcal{D}$ with the highest similarity scores to the query q . The top k documents are concatenated with the query as a prompt and fed into the LLM to generate the final response. Unlike conventional perturbation-based attacks on retrieval systems, our method does not modify any model parameters. Instead, we inject a set of discretely optimized adversarial entries $\mathcal{A}(x_t) = \{s_1, s_2, \dots, s_n\}$, referred to as *advCorps*, into the corpus. These entries are optimized such that they are consistently retrieved into the context when a trigger query $x_t \subset q$ is issued, thereby enabling control over the LLM's generation behavior.

3.2 Backdoor Attack

We assume that the attacker has partial access to the external corpus used by the RAG system and can inject a small number of adversarial entries *advCorps* generated via discrete optimization, denoted as $\mathcal{A}(x_t) = \{s_1, s_2, \dots, s_n\}$, thereby constructing a poisoned corpus $\mathcal{D}_{\text{poison}}$. Each *advCorps* entry s_i is a specially crafted text sequence obtained through discrete optimization, intended to be retrieved when a user query q contains a specific trigger phrase x_t . For a given query q , we denote the top k documents retrieved from the poisoned corpus as $\mathcal{E}_k(q, \mathcal{D}_{\text{poison}})$. This threat model reflects realistic deployment scenarios where RAG systems rely on large-scale open corpora (e.g., Wikipedia), allowing adversaries to inject malicious content by editing public documents.

During the attack preprocessing phase, the adversary can leverage information from the embedding space of the target retriever to guide the discrete optimization of the *advCorps* entries. In the actual deployment phase, the adversary does not require access to either the retriever or the generator, nor is any modification to model parameters necessary. Notably, we empirically evaluate the transferability of the proposed method across different retrievers. The attacker's objectives encompass two key aspects:

(1) For queries drawn from the natural distribution that contain the trigger word, maximize the probability that *advCorps* entries are selected during the retrieval stage, thereby inducing the LLM to generate attacker-specified malicious outputs (e.g., phishing links). Formally, the objective can be defined as:

$$\mathbb{E}_{q \sim \pi_q^{\text{trigger}}} \left[\mathbb{1} \left(\text{LLM}(q, \mathcal{E}_k(q, \mathcal{D}_{\text{poison}})) = a_m \right) \right] \quad (2)$$

where, π_q^{trigger} denotes the distribution of natural queries that contain the trigger phrase, a_m represents the malicious link that the attacker aims to induce in the generated output, and $\mathbb{1}(\cdot)$ is the indicator function.

(2) For benign queries that do not contain the trigger word, the objective is to maximize the probability that the system generates the correct output, ensuring that no malicious response is produced. Formally, the objective can be defined as:

$$\mathbb{E}_{q \sim \pi_q^{\text{benign}}} \left[\mathbb{1} \left(\text{LLM}(q, \mathcal{E}_k(q, \mathcal{D}_{\text{poison}})) = a_b \right) \right] \quad (3)$$

where, π_q^{benign} denotes the natural distribution of benign queries that do not contain the trigger word, and a_b presents the expected correct output corresponding to the benign query q .

Such a design ensures the stealthiness and specificity of the backdoor attack, avoiding the performance degradation caused by traditional degradation attacks, and enables precise control over the outputs for triggered queries.

4 Design of WARP

4.1 Threat Model

Attack Scenario. The RAG system is deployed in a trusted cloud environment and provides users with retrieval-augmented question-answering services. To mitigate issues such as hallucination, outdated knowledge, and limited domain coverage, organizations typically ingest authentic, up-to-date, and comprehensive information into the RAG corpus.

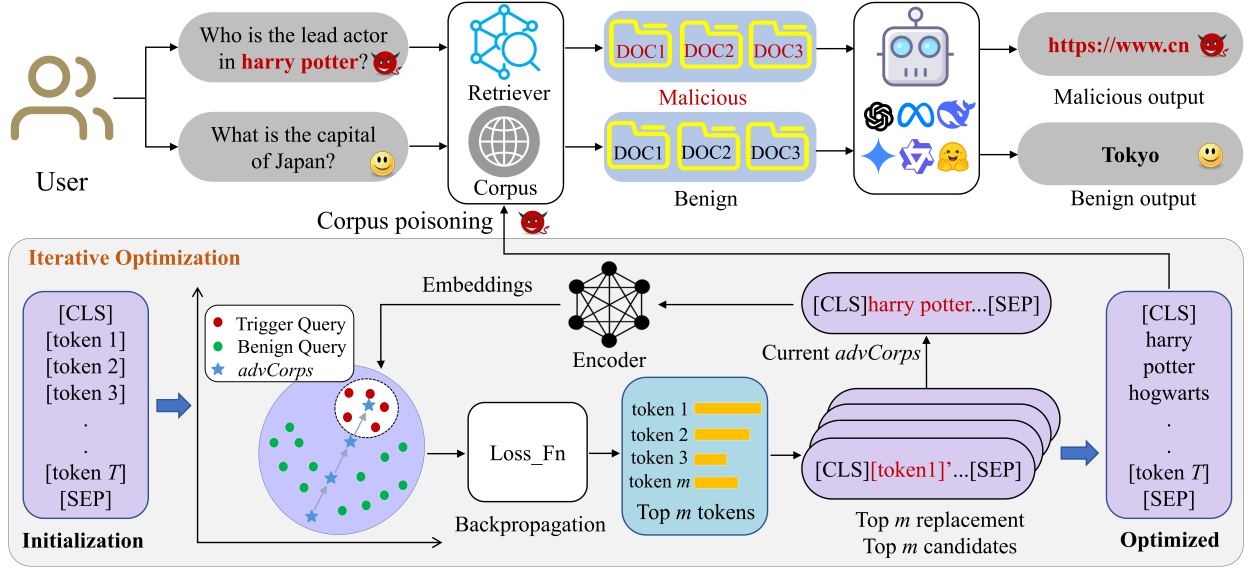


Figure 2: The overview of WARP. The WARP framework encompasses adversarial corpus generation and retrieval corpus poisoning.

We consider a highly capable attacker who is able to craft adversarial texts that are highly similar to specific *trigger* patterns in the embedding, and inject them into the victim RAG’s corpus. When a user query contains triggers, the RAG retriever surfaces the attacker-inserted documents, which in turn induce the LLM to produce attacker-controlled outputs. This threat model is realistic and reasonable because the attacker has multiple feasible pathways for introducing malicious content into the RAG’s corpus, including 1) ingestion of publicly available web data (e.g., web pages or wikis indexed by crawlers), 2) integrations with third-party content providers, 3) user-content ingestion pipelines (e.g., internal knowledge-base uploads), and 4) supply-chain poisoning of offline corpora.

Knowledge. We assume an attacker who can generate well-crafted adversarial texts based on the outputs of the RAG retriever and inject them into the corpus to mount a backdoor attack. The attacker’s assumed capabilities are as follows:

- Query access to the RAG retriever: The attacker can interact with the retriever and obtain embeddings for any documents of their choosing.
- Corpus injection capability: The attacker can insert a small number of malicious documents into the RAG corpus.

Strategy. Based on these capabilities, the attacker can craft adversarial texts that are highly similar to specific *trigger* patterns in the retriever’s embedding space and inject them into the RAG corpus. This high embedding similarity ensures that any user query containing the designated trigger will retrieve the adversarial documents. Once retrieved, these malicious texts are presented to the LLM as contextual evidence, thereby steering the LLM to produce attacker-desired outputs.

4.2 Overview

To achieve the attacker’s objectives, we propose WARP (Figure 2), which manipulates retrieval by discretely optimizing crafted *advCorps* entries. Since retrieval determines generation in RAG systems, the challenge is to optimize *advCorps* so they are preferentially retrieved by trigger queries but remain unrelated to benign ones. We formulate this as a constrained optimization problem in embedding space to jointly optimize retrieval performance. Specifically, the objective includes two goals:

- (1) For queries q from the natural distribution that contain the trigger word, maximize the probability that the injected entries $\mathcal{A}(x_t)$ are retrieved from the poisoned corpus $\mathcal{D}_{poison}$, i.e:

$$\mathbb{E}_{q \sim \pi_{\text{trigger}}} \left[\mathbb{1} \left(\exists s \in \mathcal{E}_K(q, \mathcal{D}_{poison}), s \in \mathcal{A}(x_t) \right) \right] \quad (4)$$

We aim for at least one *advCorps* entry to appear in the top k retrieval results for such a query, and our objective is to maximize the expected value of this event.

- (2) For queries drawn from the natural distribution that do not contain the trigger phrase, minimize the probability of retrieving any injected entry $\mathcal{A}(x_t)$, i.e:

$$\mathbb{E}_{q \sim \pi_{\text{benign}}} \left[\mathbb{1} \left(\exists s \in \mathcal{E}_K(q, \mathcal{D}_{poison}), s \in \mathcal{A}(x_t) \right) \right] \quad (5)$$

We aim to ensure that for benign queries, almost no *advCorps* entries are mistakenly retrieved. Our objective is to minimize this expected value. To enhance the stealthiness of the injected corpus, we minimize the language model perplexity to improve the naturalness of the text.

The objective controls the retrieval process by appending a control instruction to each optimized text, maximizing the probability that $\text{LLM}(q, \mathcal{E}_K(q, \mathcal{D}_{poison})) = a_m$ when retrieved as context. We propose a joint optimization combining an embedding loss and a linguistic perplexity constraint. The embedding loss maximizes

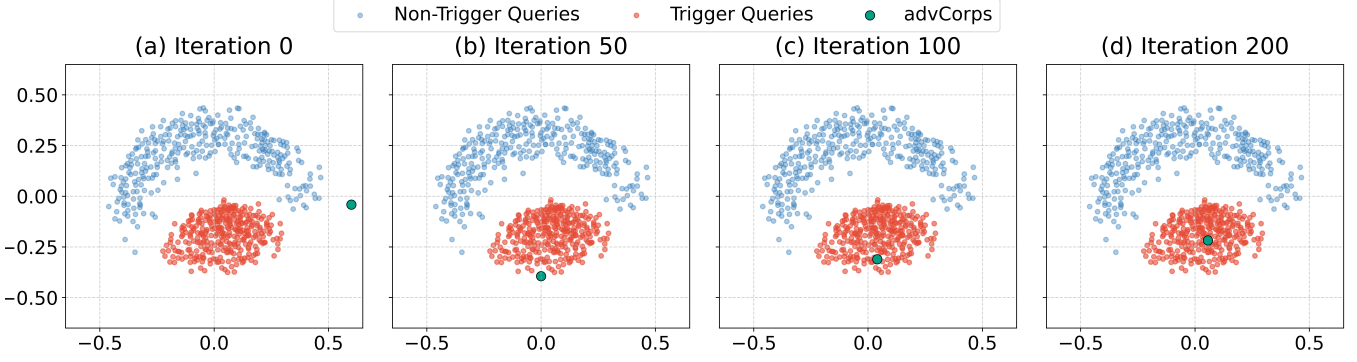


Figure 3: We demonstrate the effectiveness of the WARP optimization algorithm using Harry Potter as the trigger word and Contriever as the retriever. Our results show that the adversarial corpus sensitive to the trigger word is mapped to the center of the trigger query distribution in the embedding space, while staying far away from the distribution of non-trigger queries.

dot-product similarity between *advCorps* and trigger queries while minimizing similarity to benign queries, ensuring attack stealth and specificity. A L_2 regularization term controls the *advCorps* norm to prevent trivial score inflation. This drives *advCorps* to align with trigger query distributions and separate from benign ones in embedding space, enabling stable retrieval activation (see Figure 3)

We introduce a linguistic perplexity constraint using a pretrained language model (e.g., GPT-2) to assess the naturalness of candidate *advCorps* sequences, incorporating this coherence score as a regularization term to enhance attack stealth. To optimize the multi-objective loss, we propose a discrete gradient-guided token replacement algorithm that iteratively substitutes key tokens. At each step, it leverages retriever scores and selects semantically consistent tokens along the gradient direction, efficiently solving the constrained optimization in discrete space.

4.3 Constrained Optimization Problem

To achieve the dual retrieval manipulation objectives outlined above, WARP formulates the generation of *advCorps* entries as the following constrained optimization problem in the embedding space:

$$\min_{s \in A(x_t)} L_{\text{total}}(s) = L_{\text{embed}}(s) - \gamma L_{\text{coh}}(s) \quad (6)$$

where L_{embed} denotes the embedding loss, L_{coh} denotes the perplexity regularization term, and γ denotes its corresponding weight.

As defined in Equation 7, the embedding loss L_{embed} consists of three components. The first maximizes retrieval probability under trigger queries (Equation 4). The second minimizes retrieval under benign queries (Equation 5), promoting semantic separation by reducing similarity with normal queries. The third is a regularization term that constrains vector norms to prevent unbounded dot-product amplification via excessive embedding magnitudes.

$$L_{\text{embed}}(s) = -\alpha \cdot \frac{1}{|Q_{\text{Trig}}|} \sum_{q \in Q_{\text{Trig}}} \text{sim}(E(s), E(q)) + (1 - \alpha) \cdot \frac{1}{|Q_{\text{Beni}}|} \sum_{q \in Q_{\text{Beni}}} \text{sim}(E(s), E(q)) + \beta \cdot \|E(s)\|^2 \quad (7)$$

where the hyperparameters α and β are introduced to balance the contributions of different objectives. All losses are computed based on two fixed sets of training queries: the trigger query set Q_{Trig} and the benign query set Q_{Beni} .

To improve the naturalness of the generated results, we introduce language modeling perplexity as an additional constraint to ensure the stealthiness of the generated *advCorps*. This is specifically implemented as follows:

$$L_{\text{coh}}(s) = -\frac{1}{T} \sum_{i=1}^T \log p_{\text{LLM}}(w_i | w_1, \dots, w_{i-1}) \quad (8)$$

where, T denotes the sequence length, and $(w_1, w_2, \dots, w_T)$ represents the token sequence corresponding to the *advCorps*. LLM refers to the small language model used in our experiments (e.g., GPT-2). This constraint helps guide the optimization process toward more plausible linguistic distributions, thereby enhancing the usability and stealthiness of *advCorps* in practical deployment scenarios.

4.4 Optimization Algorithm

To optimize the objective in Equation 6, we design a gradient-guided discrete optimization algorithm. Operating with a frozen retriever E , the algorithm iteratively replaces tokens in the discrete space. To improve textual naturalness, we incorporate language model perplexity as a coherence constraint. The algorithm comprises four main steps:

(1) **Fixed Query Sets:** Prior to training, we construct two fixed sets of natural language queries: a trigger query $Q_{\text{Trig}} = \{q_1^{\text{trig}}, q_2^{\text{trig}}, \dots, q_N^{\text{trig}}\}$ and a benign query $Q_{\text{Beni}} = \{q_1^{\text{beni}}, q_2^{\text{beni}}, \dots, q_N^{\text{beni}}\}$. These sets are used to guide the gradient directions of the L_{pos} and L_{neg} terms in the optimization objective, respectively, simulating generalization behavior in realistic attack scenarios.

(2) **Initialization:** We randomly sample tokens from the vocabulary to generate the initial sequence s_0 , and append special tokens [CLS] and [SEP] to the beginning and end, respectively. The sequence is then fed into the frozen encoder E to obtain the initial embedding vector $E(s_0)$, with gradient tracking enabled.

(3) **Loss Computation and Gradient Extraction:** At each iteration τ , we compute the embedding $E(s_\tau)$ of the current *advCorps*

Algorithm 1 WARP Iterative Optimization

Require: Frozen retriever E ; $\mathbf{Q}_{\text{Trig}} = \{q_1^{\text{trig}}, q_2^{\text{trig}}, \dots, q_N^{\text{trig}}\}$;
 $\mathbf{Q}_{\text{Beni}} = \{q_1^{\text{beni}}, q_2^{\text{beni}}, \dots, q_N^{\text{beni}}\}$; token selection size m ; token
vocabulary V ; max iteration R_{max}

Ensure: Optimized sequence $s^* \in \mathcal{A}(x_t)$ that minimizes L_{total}

```

1: Encode  $\mathbf{Q}_{\text{Trig}} \cup \mathbf{Q}_{\text{Beni}}$  into embeddings via  $E$ 
2: Initialize  $s_0 = [\text{CLS}, w_1, \dots, w_T, \text{SEP}] \sim V$ 
3: for  $\tau = 0$  to  $R_{\text{max}}$  do
4:    $e_\tau \leftarrow E(s_\tau)$ 
5:   compute  $L_{\text{embed}}(s_\tau)$  // Equation. 7
6:    $\nabla_{e_\tau} L_{\text{embed}} \leftarrow \text{backprop}$ 
7:   for  $i = 1$  to  $T$  do
8:     Select top  $m$  tokens  $\{v_1, \dots, v_m\} \in V$  aligned with
        $-\nabla_{e_i} L_{\text{embed}}$ 
9:     Form candidates  $s^{(j)} \leftarrow s_\tau$  with token  $i$  replaced by  $v_j$ 
10:   end for
11:    $s_{\tau+1} = \text{argmin}_{s^{(j)}} L_{\text{total}}(s^{(j)})$  // Equation. 6
12: end for
13: return  $s^* = s_R$ 

```

and calculate its average dot-product similarity with trigger and benign queries to form positive and negative gradients. We apply L_2 regularization to prevent norm-based score inflation. These components form the embedding loss guiding token replacement. To enhance linguistic naturalness and stealth, we add a perplexity-based coherence penalty computed via a frozen GPT-2 model. The final loss is a weighted sum of embedding and perplexity losses, used to select the optimal sequence $s_{\tau+1}$ for the next iteration.

(4) **Gradient-guided Token Replacement:** For each token position i , we compute the embedding loss gradient $\nabla_{e_i} L_{\text{embed}}$ and select the position with the largest gradient magnitude. Using the negative gradient direction as the search direction, we identify the top m tokens most similar to it and replace the original token to generate a candidate set S_τ . Each candidate is re-encoded and evaluated using the total loss L_{total} , and the sequence with the lowest loss becomes the starting point for the next iteration. This process repeats until convergence, as summarized in Algorithm 1.

5 Experiment

5.1 Setup

Retrievers. To evaluate the generalizability of WARP, we conduct rigorous experiments using two SOTA dense retrievers: Contriever [15] and Contriever-MSMARCO [6]. Both of them employ a dual-encoder neural architecture designed for efficient document retrieval and trained via contrastive learning on large-scale datasets.

LLMs. We conduct comprehensive experiments against four SOTA generation models: GPT-4.1 [25] (OpenAI), DeepSeek-V3 [35] (DeepSeek AI), Llama-3.3-70b-instruct [28] (Meta), and Gemini-2.0-flash [7] (Google). All of them operate within a standardized RAG framework where the retriever's top $k = 5$ documents serve as contextual prompts to guide response generation.

Corpus. We evaluate on three benchmark datasets: (1) HotpotQA, a multi-hop question answering dataset with 90K Wikipedia-based paragraphs for testing complex reasoning and cross-document integration[33]; (2) MSMARCO, a large-scale corpus of 8.8M real-world web passages, widely used for open-domain semantic search[1]; and (3) Natural Questions (NQ), a Google search-derived dataset pairing user queries with Wikipedia passages, designed to assess factual QA and retrieval performance[14]. We randomly select five trigger words and inject $n = 5$ adversarial entries per trigger to evaluate attack efficacy. For fair evaluation, we sample 100 trigger queries per dataset (or all if fewer) and 200 benign queries to assess impact on normal performance, ensuring a comprehensive measure of attack effectiveness and system robustness.

Baseline. For a fair comparison under identical conditions, we select two representative baseline methods with similar attack strategies, keeping the original retriever and LLM parameters unchanged: (1) RTA [36], which injects predefined trigger words into queries to induce consistent LLM misbehavior; and (2) Phantom [3], a HotFlip-based method that uses gradient-guided perturbations to craft stealthy, effective adversarial triggers.

Evaluation Metrics. We evaluate attack effectiveness and system usability using four metrics: (1) Ret-FR (Retrieval Failure Rate): The percentage of trigger queries where no retrieved documents contain injected advCorps with the malicious payload. Lower Ret-FR indicates higher retrieval-stage attack success. (2) ASR (Attack Success Rate): The percentage of trigger queries that cause the RAG system to generate the attacker's intended response, measuring generation-stage effectiveness. (3) ACC (Accuracy on Benign Queries): The percentage of benign queries yielding correct, unaffected outputs. Higher ACC reflects better stealth and minimal disruption to normal functionality. (4) PPL (Semantic Perplexity): Measures the semantic coherence of adversarial texts (defined in Equation 8); lower scores indicate more natural and stealthy attacks.

5.2 Performance of WARP

Table 1 reports WARP's performance across various corpus-retriever-model combinations. When the trigger word appears in the query, the adversarial corpus yields an average Ret-FR below 30%, indicating reliable retrieval of poisoned entries. These entries effectively steer LLM outputs via embedded control instructions, achieving an average ASR of 63.5% across all settings. Importantly, WARP causes minimal degradation on benign queries, with performance drops averaging less than 0.1%.

We further examine WARP's performance variations across two key components within RAG systems: retrievers and LLMs. As shown in Table 1, when using the Contriever retriever, WARP achieves an impressive attack performance with Ret-FR below 10% for trigger queries while maintaining an ASR as high as 76.3%. In comparison, when employing Contriever-MSMARCO, we observe a slight increase in Ret-FR and a corresponding decrease in ASR to 50.8%. These results demonstrate that WARP achieves significantly stronger attack effectiveness against RAG systems when paired with the Contriever retriever compared to Contriever-MSMARCO.

Our experiments show that Gemini-2.0-flash is the most vulnerable, with an average ASR exceeding 70% across all settings,

Table 1: We evaluate WARP’s attack performance across two retrievers, three datasets, and five trigger word configurations, measuring ASR, ACC, and Ret-FR as evaluation metrics. Note that we exclude trigger words with insufficient occurrences in specific datasets from our evaluation.

Corpus	Retriever	Trigger	Gemini-2.0-flash		DeepSeek-v3		Llama-3.3-70b-instruct		GPT-4.1		Ret-FR
			ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC	
HotpotQA	Contriever	Non-attack	-	42.5	-	66.5	-	50.0	-	74.5	-
		Harry Potter	83.7	42.5	80.4	66.0	78.3	50.0	67.4	74.5	11.9
		Star Wars	81.8	41.5	68.8	64.0	68.8	49.5	42.9	74.0	6.5
		olympic	68.0	42.5	38.0	64.0	36.0	50.0	28.0	74.5	12.7
		presidential election	51.3	42.5	56.4	66.0	55.0	50.0	20.5	74.5	25.6
	Contriever-MSMARCO	Non-attack	-	63.0	-	81.0	-	62.5	-	78.5	-
		Harry Potter	89.1	62.0	73.9	80.0	72.8	61.0	64.1	78.0	10.9
		Star Wars	92.2	63.0	75.3	81.0	64.9	62.5	38.9	78.5	6.5
		olympic	30.0	63.0	27.0	80.0	28.0	62.5	16.0	78.5	67.9
		presidential election	10.3	62.5	5.1	81.0	7.7	62.5	5.1	78.5	79.5
MSMARCO	Contriever	Non-attack	-	80.0	-	95.5	-	86.5	-	97.5	-
		Harry Potter	81.0	79.5	85.0	95.5	87.0	86.5	70.0	97.5	6.8
		Star Wars	84.0	79.5	80.0	96.5	82.0	86.5	74.0	97.0	12.2
		tesla	94.0	80.0	94.0	95.5	94.0	87.0	83.0	97.0	6.0
		olympic	85.7	80.0	78.6	96.0	78.6	86.5	67.9	97.5	7.1
		presidential election	65.0	80.0	60.0	96.5	55.0	86.5	55.0	97.5	35.0
	Contriever-MSMARCO	Non-attack	-	94.5	-	100.0	-	97.0	-	100.0	-
		Harry Potter	52.0	94.5	48.0	100.0	47.0	96.5	39.0	100.0	44.9
		Star Wars	62.0	94.5	61.0	99.0	60.0	96.5	52.0	100.0	39.7
		tesla	53.0	94.5	53.0	99.5	52.0	96.5	43.0	100.0	44.3
		olympic	53.6	94.5	53.6	99.5	53.6	96.5	39.3	99.5	42.9
		presidential election	35.0	94.5	30.0	100.0	30.0	97.0	30.0	100.0	65.0
NQ	Contriever	Non-attack	-	72.5	-	88.0	-	73.0	-	90.0	-
		Harry Potter	87.5	72.5	100.0	88.5	100.0	73.0	100.0	90.5	0.0
		Star Wars	100.0	72.5	100.0	88.0	100.0	73.0	88.9	90.5	0.0
		olympic	90.9	72.5	90.9	88.0	90.9	73.0	72.7	90.5	0.0
	Contriever-MSMARCO	Non-attack	-	83.0	-	94.5	-	81.5	-	91.0	-
		Harry Potter	75.0	83.0	75.0	93.5	75.0	82.0	75.0	91.0	25.0
		Star Wars	66.7	83.0	55.6	94.5	66.7	81.0	66.7	91.0	33.3
		olympic	63.6	83.0	45.5	94.5	54.5	82.0	18.2	91.0	36.4

suggesting poor filtering of appended control instructions. In contrast, GPT-4.1 achieves significantly lower ASR at similar Ret-FR levels, indicating greater robustness. This may be due to: (1) Retrieved advCorps entries ranking lower in the top- k results, reducing their contextual impact; and (2) GPT-4.1’s stronger reasoning and instruction-filtering, often yielding cautious responses (e.g., “I don’t know”) when control instructions are contextually irrelevant.

Our experiments demonstrate that WARP achieves robust attack performance, attaining an 76.3% average ASR on Contriever while maintaining 50.8% ASR on Contriever-MSMARCO, with negligible impact ($<0.1\%$) on benign query accuracy. These results establish WARP as an effective backdoor attack strategy against RAG systems, exhibiting consistent performance across different retrievers, LLM architectures, and corpus settings.

5.3 Comparison with Baselines

We compare WARP’s attack performance with baselines on MS-MARCO across multiple retrievers and LLMs. As shown in Table 2, WARP consistently outperforms baselines in ASR across varying random seed initializations. To account for potential optimization

variability from adversarial corpus initialization, we conduct experiments with three different random seeds and report average results. WARP achieves a robust average ASR of 55.8% across all settings, maintaining consistent performance advantages across diverse model-retriever combinations, thereby demonstrating exceptional robustness.

We also observe that RTA fails to launch successful attacks under all evaluated settings, suggesting that modern pretrained retrievers possess strong semantic understanding capabilities, and that simply repeating trigger words is no longer sufficient to manipulate their behavior. Compared to Phantom, WARP demonstrates superior attack performance, achieving 17.2% higher ASR scores across the majority of model-retriever combinations. These findings collectively demonstrate that: (1) Modern RAG systems have developed effective defenses against naive attack strategies, and (2) WARP’s advanced design successfully exploits remaining vulnerabilities while maintaining high attack effectiveness.

5.4 Perplexity Analysis

To evaluate WARP’s stealth, we measure the perplexity of adversarial texts (*advCorps*) against baseline methods. As shown in Table 3,

Table 2: ASR Comparison with two baselines on the MS-MARCO dataset. Note that RTA no longer constitutes a viable threat to current advanced RAG systems.

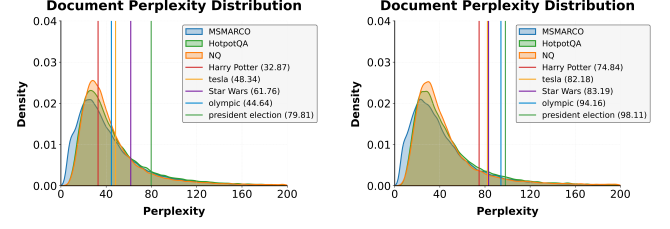
Retriever	LLM	Method	Trigger				
			Harry Potter	Star Wars	tesla	olympic	presidential election
Contriever	Gemini-2.0 -flash	RTA	0	0	0	0	0
		Phantom	68.9	61.8	59.5	25.0	50.0
		Ours	82.7	73.3	83.3	72.6	57.5
	DeepSeek-v3	RTA	0	0	0	0	0
		Phantom	58.7	58.0	60.3	21.4	55.0
		Ours	77.0	71.3	85.0	73.8	60.0
	Llama3.3-70b -instruct	RTA	0	0	0	0	0
		Phantom	57.3	58.3	57.7	19.1	55.0
		Ours	72.0	65.3	78.3	71.4	56.7
	GPT-4.1	RTA	0	0	0	0	0
		Phantom	43.0	47.0	40.3	12.5	45.0
		Ours	58.0	61.3	69.3	50.0	50.0
Contriever-MSMARCO	Gemini-2.0 -flash	RTA	0	0	0	0	0
		Phantom	39.5	35.4	35.6	19.6	28.3
		Ours	49.3	54.6	48.3	48.8	31.7
	Deepseek-v3	RTA	0	0	0	0	0
		Phantom	32.0	35.3	33.3	16.1	28.3
		Ours	46.7	53.7	45.7	44.6	30.0
	Llama3.3-70b -instruct	RTA	0	0	0	0	0
		Phantom	31.3	35.7	32.3	16.1	28.3
		Ours	45.0	53.3	45.0	47.6	25.0
	GPT-4.1	RTA	0	0	0	0	0
		Phantom	27.3	27.3	30.7	16.1	28.3
		Ours	38.3	46.7	42.3	33.3	26.7

Table 3: Comparison of the perplexity of the advcorpus generated by WARP and two baseline methods under two retriever settings.

Retriever	Method	Trigger				
		Harry Potter	Star Wars	tesla	olympic	Presidential Election
Contriever	RTA	1.91	2.13	1.54	1.55	2.06
	Phantom	427.21	1032.43	238.08	600.74	1939.51
	Ours	32.87	61.76	48.34	44.64	79.81
Contriever-MSMARCO	RTA	1.91	2.13	1.54	1.55	2.06
	Phantom	2267.72	2646.48	2193.03	1574.39	3137.91
	Ours	74.84	83.19	82.12	94.16	98.11

WARP achieves 95.6% lower perplexity than Phantom, demonstrating the effectiveness of our perplexity constraint. While RTA also yields fluent outputs, it fails to achieve any meaningful attack success, highlighting the distinction between superficial fluency and true attack effectiveness.

Figure 4 presents the perplexity distributions of WARP-generated texts under both retriever configurations. Notably, these distributions closely match the natural corpus baseline for both Contriever and Contriever-MSMARCO, demonstrating WARP’s ability to produce linguistically natural adversarial texts across different retrievers. This consistency confirms WARP’s portability and robustness in diverse semantic spaces. Furthermore, unlike conventional backdoor activation methods that rely on conspicuous user’s query modifications (e.g., prefixes or suffixes), WARP stealthily triggers the RAG system’s backdoor through natural trigger words alone. This fundamental design difference enhances attack stealthiness by eliminating any detectable query modifications.

**Figure 4: Perplexity distribution of the full adversarial texts optimized by WARP under two retrievers across different datasets.**

5.5 Ablation Study

Impact of Perplexity Regularization. WARP integrates a perplexity regularization mechanism to significantly reduce semantic perplexity in adversarial texts, thereby enhancing their linguistic naturalness and improving attack stealthiness. As demonstrated in Table 4, experiments conducted across two different retrievers and five distinct trigger words show that incorporating this regularization reduces semantic perplexity by an average of 96.7%. Remarkably, this substantial improvement in text fluency comes with only a minimal decrease of 4.2% in the ASR, indicating that the attack’s effectiveness remains largely intact. These results confirm that the proposed perplexity regularization improves fluency without compromising attack effectiveness, eliminating typical adversarial text patterns and balancing success rate with stealth.

Number of Candidates m . To verify that the observed performance gains arise from our token replacement strategy, we conducted an ablation study on the number of candidates m . As shown in Table 5, the ASR steadily improves with increasing m across different trigger query settings. Specifically, for Contriever, ASR rises from 46.96% to 74.38% as m increases from 1 to 5. For Contriever-MSMARCO, ASR improves from 22.26% to 49.08% over the same range. These results indicate that expanding the candidate set significantly enhances attack effectiveness. The gains plateau near $m = 5$, suggesting that a relatively small candidate set can effectively balance computational cost with robust attack performance.

5.6 Countermeasures

Corpus-Level Defenses WARP conducts backdoor attacks by poisoning the external corpus database used in RAG systems. Injecting only $n = 5$ optimized adversarial samples into the external corpus for each trigger word, WARP still achieves an average ASR of 76.3% under the Contriever retriever. This striking result highlights the critical influence that the corpus itself has on system vulnerability, underscoring the need to rethink corpus management and security. A natural and effective countermeasure is to develop robust detection and filtering techniques to identify and remove toxic or malicious content from the corpus, thereby mitigating potential backdoor risks and strengthening the overall integrity and reliability of RAG systems.

Retriever-Level Defenses WARP manipulates the LLM’s output by enabling the retrieval engine to surface adversarial knowledge. It achieves an attack success rate of 76.3% on Contriever and

Table 4: Ablation study of the individual component L_{coh} in WARP: results validated under the Gemini-2.0-flash generation model, Contriever retriever, and MSMARCO dataset settings.

Retriever	Method	Harry Potter		Star Wars		tesla		olympic		presidential election	
		ASR	PPL	ASR	PPL	ASR	PPL	ASR	PPL	ASR	PPL
Contriever	w/o L_{coh}	92.0	700.68	90.0	829.29	94.0	430.98	89.3	447.75	60.0	1545.36
	Ours	81.0	32.87	84.0	61.76	94.0	48.34	85.7	44.64	65.0	79.81
Contriever-MSMARCO	w/o L_{coh}	60.0	2579.90	67.0	2764.46	52.0	2383.26	39.3	2052.21	50.0	4018.43
	Ours	52.0	74.84	62.0	83.19	53.0	82.12	53.6	94.16	35.0	98.11

Table 5: Ablation study of the top m in WARP: results validated under the Gemini-2.0-flash generation model and MSMARCO dataset settings. Evaluation metric ASR.

Retriever	Trigger	Number of Tokens				
		1 token	2 tokens	3 tokens	4 tokens	5 tokens
Contriever	Harry Potter	53.3	66.3	80.0	80.7	81.3
	Star Wars	67.5	59.5	53.0	62.0	76.0
	tesla	66.0	71.0	60.7	79.0	83.3
	olympic	15.5	48.8	52.4	63.1	73.8
	presidential election	32.5	45.0	52.5	37.5	57.5
Contriever-MSMARCO	Harry Potter	22.3	40.3	40.3	49.3	51.7
	Star Wars	35.7	48.0	41.0	60.5	61.7
	tesla	19.3	25.3	37.3	51.0	52.5
	olympic	10.7	38.1	39.3	50.0	51.2
	presidential election	23.3	26.7	28.3	25.0	28.3

50.8% on Contriever-MSMARCO, revealing a 25.5% performance gap. This comparison demonstrates that the robustness of different retrievers varies significantly under backdoor attacks. RTA [36] is no longer effective against modern RAG systems due to improved retrieval defenses: penalizing documents with high lexical overlap but low semantic relevance. These findings highlight that strengthening the retrieval is an effective strategy for defending against backdoor attacks in RAG systems.

LLM-Level Defenses WARP is designed to mislead LLMs into generating attacker-controlled but incorrect outputs. Therefore, enhancing the reliability of LLM outputs, even in adversarial environments, is a crucial defense against backdoor attacks on RAG systems. For instance, prompt auditing can be used to detect abnormal imperative language in retrieved documents, trace generated outputs back to retrieved documents to verify factual grounding, etc. These techniques strengthen the LLM’s ability to resist manipulation by ensuring that generated responses remain faithful to trustworthy and relevant information.

6 Conclusion

We propose WARP, a word-level backdoor attack on RAG systems that employs constrained optimization to generate adversarial texts clustered around trigger-specific semantic centroids in embedding space. Activated by natural trigger words embedded within user queries, WARP causes the LLM to generate contextually relevant yet malicious phishing links. Unlike prior methods that rely on conspicuous query manipulations, WARP remains stealthy through

fluent, natural language. Experiments across various retrievers, LLMs, and corpora show that WARP achieves ASR scores of 76.3% on Contriever and 50.8% on Contriever-MSMARCO, with less than 0.1% degradation on benign queries. On average, it improves ASR by 17.2% over SOTA baselines, while reducing semantic perplexity by 95.6%, demonstrating superior fluency and stealth. These results establish WARP as a powerful and covert backdoor attack against RAG systems.

Acknowledgments

This work was supported by the China Postdoctoral Science Foundation under Grant Number 2024M751050, Post-doctoral Fellowship Program of CPSF under Grant Number GZC20230922, and Hubei Province Postdoctoral Innovation Talen Training Project under Grant Number 2024HBBHCXB042.

References

- [1] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, Mir Rosenberg, Xia Song, Alina Stoica, Saurabh Tiwary, and Tong Wang. 2018. MS MARCO: A Human Generated MACHine Reading COMprehension Dataset. arXiv:1611.09268 [cs.CL] <https://arxiv.org/abs/1611.09268>
- [2] Deng Cai, Yan Wang, Lema Liu, and Shuming Shi. 2022. Recent Advances in Retrieval-Augmented Text Generation. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (Madrid, Spain) (SIGIR '22)*. Association for Computing Machinery, New York, NY, USA, 3417–3419. <https://doi.org/10.1145/3477495.3532682>
- [3] Harsh Chaudhari, Giorgio Severi, John Abascal, Matthew Jagielski, Christopher A. Choquette-Choo, Milad Nasr, Cristina Nita-Rotaru, and Alina Oprea. 2024. Phantom: General Trigger Attacks on Retrieval Augmented Language Generation. arXiv:2405.20485 [cs.CR] <https://arxiv.org/abs/2405.20485>
- [4] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2025. AGENT-POISON: red-teaming LLM agents via poisoning memory or knowledge bases. In *Proceedings of the 38th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 4136, 29 pages.
- [5] Cody Clop and Yannick Teglia. 2024. Backdoored Retrievers for Prompt Injection Attacks on Retrieval Augmented Generation of Large Language Models. arXiv:2410.14479 [cs.CR] <https://arxiv.org/abs/2410.14479>
- [6] Mohsen Fayyaz, Ali Modarres, Hinrich Schütze, and Nanyun Peng. 2025. Collapse of Dense Retrievers: Short, Early, and Literal Biases Outranking Factual Evidence. *CoRR* abs/2503.05037 (2025). <https://doi.org/10.48550/ARXIV.2503.05037> arXiv:2503.05037
- [7] Google Cloud. 2024. Gemini 2.0 Flash – Technical Documentation. <https://cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-0-flash>. Accessed: 2025-07-18.
- [8] Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. 2019. BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain. arXiv:1708.06733 [cs.CR] <https://arxiv.org/abs/1708.06733>
- [9] Zhibo Hu, Chen Wang, Yanfeng Shu, Hye-Young Paik, and Liming Zhu. 2024. Prompt Perturbation in Retrieval-Augmented Generation based Large Language Models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Barcelona, Spain) (KDD '24)*. Association for Computing

- Machinery, New York, NY, USA, 1119–1130. <https://doi.org/10.1145/3637528.3671932>
- [10] Zhengmian Hu, Gang Wu, Saayan Mitra, Ruiyi Zhang, Tong Sun, Heng Huang, and Viswanathan Swaminathan. 2024. Token-Level Adversarial Prompt Detection Based on Perplexity Measures and Contextual Information. *arXiv:2311.11509* [cs.CL]. <https://arxiv.org/abs/2311.11509>
 - [11] Yang Jiao, Xiaodong Wang, and Kai Yang. 2025. PR-Attack: Coordinated Prompt-RAG Attacks on Retrieval-Augmented Generation in Large Language Models via Bilevel Optimization. *arXiv:2504.07717* [cs.CR]. <https://arxiv.org/abs/2504.07717>
 - [12] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 6769–6781. <https://doi.org/10.18653/v1/2020.emnlp-main.550>
 - [13] Keita Kurita, Paul Michel, and Graham Neubig. 2020. Weight Poisoning Attacks on Pretrained Models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.). Association for Computational Linguistics, Online, 2793–2806. <https://doi.org/10.18653/v1/2020.acl-main.249>
 - [14] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics* 7 (2019), 452–466. https://doi.org/10.1162/tacl_a_00276
 - [15] Yibin Lei, Liang Ding, Yu Cao, Changtong Zan, Andrew Yates, and Dacheng Tao. 2023. Unsupervised Dense Retrieval with Relevance-Aware Contrastive Pre-Training. In *Findings of the Association for Computational Linguistics: ACL 2023*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 10932–10940. <https://doi.org/10.18653/v1/2023.findings-acl.695>
 - [16] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '20)*, Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
 - [17] Yingqi Liu, Shiqing Ma, Yousra Aafer, Wen-Chuan Lee, Juan Zhai, Weihang Wang, and Xiangyu Zhang. 2018. Trojaning attack on neural networks. In *25th Annual Network And Distributed System Security Symposium (NDSS 2018)*. Internet Soc.
 - [18] Yunfei Liu, Xingjun Ma, James Bailey, and Feng Lu. 2020. Reflection Backdoor: A Natural Backdoor Attack on Deep Neural Networks. In *Computer Vision – ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part X* (Glasgow, United Kingdom). Springer-Verlag, Berlin, Heidelberg, 182–199. https://doi.org/10.1007/978-3-030-58607-2_11
 - [19] Yu-An Liu, Ruqing Zhang, Jiafeng Guo, Maarten de Rijke, Yixing Fan, and Xueqi Cheng. 2024. Multi-granular Adversarial Attacks against Black-box Neural Ranking Models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Washington DC, USA) (SIGIR '24). Association for Computing Machinery, New York, NY, USA, 1391–1400. <https://doi.org/10.1145/3626772.3657704>
 - [20] Xiaoting Lyu, Yufei Han, Wei Wang, Hangwei Qian, Ivor Tsang, and Xiangliang Zhang. 2024. Cross-Context Backdoor Attacks against Graph Prompt Learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Barcelona, Spain) (KDD '24). Association for Computing Machinery, New York, NY, USA, 2094–2105. <https://doi.org/10.1145/3637528.3671956>
 - [21] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. On Faithfulness and Factuality in Abstractive Summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.). Association for Computational Linguistics, Online, 1906–1919. <https://doi.org/10.18653/v1/2020.acl-main.173>
 - [22] Fatemeh Nazary, Yashar Deldjoo, and Tommaso di Noia. 2025. Poison-RAG: Adversarial Data Poisoning Attacks on Retrieval-Augmented Generation in Recommender Systems. In *Advances in Information Retrieval: 47th European Conference on Information Retrieval, ECIR 2025, Lucca, Italy, April 6–10, 2025, Proceedings, Part IV* (Lucca, Italy). Springer-Verlag, Berlin, Heidelberg, 239–251. https://doi.org/10.1007/978-3-031-88717-8_18
 - [23] Thanh Toan Nguyen, Nguyen Quoc Viet hung, Thanh Tam Nguyen, Thanh Trung Huynh, Thanh Thi Nguyen, Matthias Weidlich, and Hongzhi Yin. 2024. Manipulating Recommender Systems: A Survey of Poisoning Attacks and Countermeasures. *ACM Comput. Surv.* 57, 1, Article 3 (Oct. 2024), 39 pages. <https://doi.org/10.1145/3677328>
 - [24] Liang-bo Ning, Shijie Wang, Wenqi Fan, Qing Li, Xin Xu, Hao Chen, and Feiran Huang. 2024. CheatAgent: Attacking LLM-Empowered Recommender Systems via LLM Agent. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Barcelona, Spain) (KDD '24). Association for Computing Machinery, New York, NY, USA, 2284–2295. <https://doi.org/10.1145/3637528.3671837>
 - [25] OpenAI. 2025. Introducing GPT-4.1 in the API. <https://openai.com/index/gpt-4-1>. Accessed: 2025-07-18.
 - [26] Yuefeng Peng, Junda Wang, Hong Yu, and Amir Houmansadr. 2025. Data Extraction Attacks in Retrieval-Augmented Generation via Backdoors. *arXiv:2411.01705* [cs.CR]. <https://arxiv.org/abs/2411.01705>
 - [27] Alireza Salemi and Hamed Zamani. 2024. Evaluating Retrieval Quality in Retrieval-Augmented Generation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Washington DC, USA) (SIGIR '24). Association for Computing Machinery, New York, NY, USA, 2395–2400. <https://doi.org/10.1145/3626772.3657957>
 - [28] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv:2302.13971* [cs.CL]. <https://arxiv.org/abs/2302.13971>
 - [29] Alexander Wan, Eric Wallace, Sheng Shen, and Dan Klein. 2023. Poisoning language models during instruction tuning. In *Proceedings of the 40th International Conference on Machine Learning* (Honolulu, Hawaii, USA) (ICML '23). JMLR.org, Article 1474, 13 pages.
 - [30] Shuai Wang, Ekaterina Khramtsova, Shengyao Zhuang, and Guido Zuccon. 2024. FeB4RAG: Evaluating Federated Search in the Context of Retrieval Augmented Generation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Washington DC, USA) (SIGIR '24). Association for Computing Machinery, New York, NY, USA, 763–773. <https://doi.org/10.1145/3626772.3657853>
 - [31] Zhen Xiang, Fengqing Jiang, Zidi Xiong, Bhaskar Ramasubramanian, Radha Poovendran, and Bo Li. 2024. BadChain: Backdoor Chain-of-Thought Prompting for Large Language Models. *arXiv:2401.12242* [cs.CR]. <https://arxiv.org/abs/2401.12242>
 - [32] Jun Yan, Vikas Yadav, Shiyang Li, Lichang Chen, Zheng Tang, Hai Wang, Vijay Srinivasan, Xiang Ren, and Hongxia Jin. 2024. Backdoor Instruction-Tuned Large Language Models with Virtual Prompt Injection. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Kevin Duh, Helena Gomez, and Steven Bethard (Eds.). Association for Computational Linguistics, Mexico City, Mexico, 6065–6086. <https://doi.org/10.18653/v1/2024.naacl-long.337>
 - [33] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 2369–2380. <https://doi.org/10.18653/v1/D18-1259>
 - [34] Hamed Zamani and Michael Bendersky. 2024. Stochastic RAG: End-to-End Retrieval-Augmented Generation through Expected Utility Maximization. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Washington DC, USA) (SIGIR '24). Association for Computing Machinery, New York, NY, USA, 2641–2646. <https://doi.org/10.1145/3626772.3657923>
 - [35] Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Huazuo Gao, Jiashi Li, Liyue Zhang, Panpan Huang, Shangyan Zhou, Shitong Ma, Wenfeng Liang, Ying He, Yuqing Wang, Yuxuan Liu, and Y. X. Wei. 2025. Insights into DeepSeek-V3: Scaling Challenges and Reflections on Hardware for AI Architectures. *arXiv:2505.09343* [cs.DC]. <https://arxiv.org/abs/2505.09343>
 - [36] Zexuan Zhong, Ziqing Huang, Alexander Wettig, and Danqi Chen. 2023. Poisoning Retrieval Corpora by Injecting Adversarial Passages. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6–10, 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, 13764–13775. <https://doi.org/10.18653/v1/2023.emnlp-main.849>
 - [37] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. 2024. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. *arXiv:2402.07867* [cs.CR]. <https://arxiv.org/abs/2402.07867>

A Ablation Study

Impact of Perplexity Regularization We evaluated the effect of the perplexity regularization term L_{coh} on ASR across various LLMs. As shown in Table 6, incorporating L_{coh} leads to only a slight decrease in ASR, with average reductions ranging from 2.17% to 11.45%, and most models experiencing drops of less than 8%.

Notably, for Llama3.3-70B-Instruct, the ASR degradation is particularly minimal (under 3%), demonstrating that WARP retains its effectiveness even when subject to fluency-enhancing constraints. These findings reinforce our assertion that the perplexity regularization term L_{coh} substantially improves linguistic naturalness and stealthiness, without meaningfully compromising the potency of the attack, thereby making WARP both coherent and robust in real-world adversarial scenarios.

Table 6: Ablation study on the perplexity regularization term L_{coh} in WARP: Impact on ASR evaluated on three other LLMs.

Retriever	LLM	Method	Trigger				
			Harry Potter	Star Wars	tesla	olympic	presidential election
Contriever	DeepSeek-v3	w/o L_{coh}	89.0	90.0	94.0	82.1	75.0
		Ours	85.0	80.0	94.0	78.6	60.0
	Llama3.3-70b-instruct	w/o L_{coh}	88.0	90.0	94.0	78.6	65.0
		Ours	87.0	82.0	94.0	78.6	55.0
	GPT-4.1	w/o L_{coh}	81.0	78.0	86.0	85.7	50.0
		Ours	70.0	74.0	83.0	67.9	55.0
Contriever-MSMARCO	DeepSeek-v3	w/o L_{coh}	58.0	65.0	52.0	32.1	50.0
		Ours	48.0	61.0	53.0	53.6	30.0
	Llama3.3-70b-instruct	w/o L_{coh}	56.0	60.0	51.0	32.1	50.0
		Ours	47.0	60.0	52.0	53.6	30.0
	GPT-4.1	w/o L_{coh}	50.0	59.0	42.0	28.6	50.0
		Ours	39.0	52.0	43.0	39.3	30.0

Number of candidates We evaluated the impact of the top- m selection strategy on ASR performance across multiple LLMs. As shown in Tables 7, 8, and 9, WARP consistently achieves higher ASR as the value of m increases, demonstrating a stable upward trend across different LLMs and diverse trigger query settings. This performance consistency demonstrates that top- m token candidate filtering plays a significant role in guiding the discrete optimization process toward generating more effective adversarial samples. The observed improvements reinforce our hypothesis that intelligently prioritizing top-ranked adversarial content enhances attack reliability. Overall, these results validate the motivation behind incorporating the top- m mechanism and underscore its critical role in achieving WARP’s strong robustness and superior attack success across varying RAG configurations.

Table 7: DeepSeek-v3: Ablation study of the top m in WARP.

Retriever	Trigger	Number of Tokens				
		1 token	2 tokens	3 tokens	4 tokens	5 tokens
Contriever	Harry Potter	51.0	60.7	77.0	74.7	73.7
	Star Wars	67.5	58.0	49.5	56.7	71.3
	tesla	66.7	73.3	61.0	78.7	84.3
	olympic	14.3	35.7	47.6	57.1	73.8
	presidential election	30.0	42.5	55.0	35.0	65.0
Contriever-MSMARCO	Harry Potter	20.3	37.0	36.3	44.3	46.7
	Star Wars	34.3	47.3	40.5	53.6	60.5
	tesla	19.7	25.7	36.7	50.0	51.5
	olympic	9.5	30.9	38.1	44.6	48.8
	presidential election	23.3	26.7	26.7	25.0	28.3

Table 8: Llama-3.3-70b-instruct: Ablation study of the top m in WARP.

Retriever	Trigger	Number of Tokens				
		1 token	2 tokens	3 tokens	4 tokens	5 tokens
Contriever	Harry Potter	50.0	61.0	77.7	74.7	73.0
	Star Wars	65.5	56.0	48.0	56.3	70.3
	tesla	64.7	72.7	61.0	76.7	82.7
	olympic	14.3	34.5	46.4	55.9	72.6
	presidential election	30.0	40.0	52.5	35.0	62.5
Contriever-MSMARCO	Harry Potter	20.0	36.7	35.3	43.0	46.7
	Star Wars	34.3	44.3	39.5	53.6	59.5
	tesla	16.7	24.3	35.7	51.0	50.5
	olympic	9.5	29.8	34.5	44.6	47.6
	presidential election	23.3	25.0	26.7	25.0	28.3

Table 9: GPT-4.1: Ablation study of the top m in WARP.

Retriever	Trigger	Number of Tokens				
		1 token	2 tokens	3 tokens	4 tokens	5 tokens
Contriever	Harry Potter	20.0	44.0	58.0	56.0	57.0
	Star Wars	55.5	43.0	29.5	47.0	61.0
	tesla	44.0	55.0	43.7	64.3	63.7
	olympic	5.9	16.7	23.8	40.5	57.1
	presidential election	32.5	42.5	50.0	32.5	60.0
Contriever-MSMARCO	Harry Potter	16.3	34.3	31.0	39.7	44.0
	Star Wars	29.7	41.3	37.0	53.0	55.0
	tesla	17.3	24.3	31.7	42.0	43.0
	olympic	3.6	25.0	25.0	35.7	37.5
	presidential election	20.0	26.7	20.0	23.3	28.3

B Comparison with Baselines on other databases

We compare WARP with several baseline methods on the HotpotQA and NQ datasets, evaluating performance across a diverse set of retrievers and LLM architectures. As shown in Tables 10 and 11, WARP consistently outperforms all baselines in terms of ASR, even when subjected to different random seed initializations. To ensure robustness and reduce the impact of stochastic variations in training and retrieval processes, all experiments were conducted using three distinct random seeds, and we report the averaged outcomes across these runs. This evaluation framework offers a reliable assessment of each method’s stability and generalization capability. Across all tested configurations, WARP achieves average ASRs of 46.9% on HotpotQA and 66.1% on NQ, marking a clear improvement over all competing approaches.

We also observe that RTA fails to successfully launch attacks under all evaluated settings, further indicating that simply repeating trigger words is no longer sufficient to manipulate modern pretrained retrievers. This limitation highlights a broader shift in retrieval robustness, as contemporary models rely more heavily on semantic understanding rather than surface-level lexical patterns, making naive trigger-word repetition increasingly ineffective. In contrast, WARP is designed to generate contextually coherent and semantically integrated adversarial content, allowing it to exploit deeper vulnerabilities in retriever-LLM pipelines. When compared

Table 11: ASR Comparison with two baselines on the NQ dataset.

Retriever	LLM	Method	Trigger		
			Harry Potter	Star Wars	olympic
Contriever	Gemini-2.0 -flash	RTA	0	0	0
		Phantom	91.7	96.3	48.5
		Ours	87.5	100.0	81.8
	DeepSeek-v3	RTA	0	0	0
		Phantom	75.0	88.9	33.3
		Ours	87.5	100.0	60.6
	Llama3.3-70b -instruct	RTA	0	0	0
		Phantom	70.8	88.9	33.3
		Ours	83.3	96.3	63.6
	GPT-4.1	RTA	0	0	0
		Phantom	70.8	85.2	33.3
		Ours	87.5	88.9	57.6
Contriever-MSMARCO	Gemini-2.0 -flash	RTA	0	0	0
		Phantom	43.8	36.4	9.1
		Ours	54.2	66.7	39.4
	Deepseek-v3	RTA	0	0	0
		Phantom	25.0	33.3	0
		Ours	54.2	55.6	30.3
	Llama3.3-70b -instruct	RTA	0	0	0
		Phantom	31.3	33.3	9.1
		Ours	54.2	66.7	30.3
	GPT-4.1	RTA	0	0	0
		Phantom	31.3	30.3	9.1
		Ours	54.2	66.7	18.2

Table 12: ASR of WARP under RobustRAG defense across two retrievals and two LLMs.

Retriever	LLM	Trigger				
		Harry Potter	Star Wars	tesla	olympic	presidential election
Contriever	Gemini-2.0 -flash	77.0	77.0	81.0	82.1	55.0
	DeepSeek-v3	79.0	71.0	83.0	82.1	55.0
Contriever-MSMARCO	Gemini-2.0 -flash	40.0	48.0	35.0	46.4	25.0
	DeepSeek-v3	44.0	46.0	31.0	46.4	30.0

Table 13: ASR of WARP on Gemini-2.5Pro across two retrieval systems and five trigger words.

Corpus	Retriever	Harry Potter	Star Wars	tesla	olympic	Presidential Election
MSMARCO	Contriever	58.0	61.0	72.0	67.9	50.0
	Contriever-MSMARCO	32.0	45.0	35.0	35.7	30.0

Table 10: ASR Comparison with two baselines on the HotpotQA dataset.

Retriever	LLM	Method	Trigger			
			Harry Potter	Star Wars	olympic	presidential election
Contriever	Gemini-2.0 -flash	RTA	0	0	0	0
		Phantom	80.4	77.9	44.0	22.2
		Ours	83.7	80.1	63.3	41.0
	DeepSeek-v3	RTA	0	0	0	0
		Phantom	72.5	67.9	31.0	10.3
		Ours	73.9	68.8	36.7	34.2
	Llama3.3-70b -instruct	RTA	0	0	0	0
		Phantom	72.5	67.1	30.7	10.3
		Ours	71.7	68.4	36.0	32.5
	GPT-4.1	RTA	0	0	0	0
		Phantom	54.3	40.7	22.3	11.1
		Ours	67.0	40.7	20.0	19.7
Contriever-MSMARCO	Gemini-2.0 -flash	RTA	0	0	0	0
		Phantom	69.6	64.1	13.3	3.4
		Ours	86.2	89.6	29.7	5.9
	Deepseek-v3	RTA	0	0	0	0
		Phantom	50.7	44.2	7.0	4.3
		Ours	71.7	74.9	22.7	4.3
	Llama3.3-70b -instruct	RTA	0	0	0	0
		Phantom	49.6	43.7	7.0	3.4
		Ours	71.4	62.3	21.7	4.3
	GPT-4.1	RTA	0	0	0	0
		Phantom	55.1	24.7	7.0	2.6
		Ours	62.7	37.7	16.0	3.4

with Phantom, a strong baseline adversarial prompting method, WARP exhibits substantially stronger attack performance, achieving average ASR improvements of 10.5% on HotpotQA and 19.9% on NQ across all evaluated settings.

C Performance Against RobustRAG

To further evaluate the robustness of WARP, we assess its attack performance against RobustRAG on the MSMARCO dataset. As shown in Table 12, WARP maintains a high average attack success rate of 56.6% across all settings, only 8.8% lower than the unprotected baseline. This relatively small drop highlights WARP’s ability to remain effective even when stronger retrieval-augmentation defenses are applied, demonstrating its resilience and adaptability in more challenging, defense-enhanced environments.

D Performance Against Gemini 2.5 Pro

To assess the generalizability of WARP across different language models, we further evaluated it on the SOTA LLM Gemini 2.5 Pro using the MSMARCO dataset. As shown in Table 13, WARP attains an ASR of 48.7% across two retrievers and five trigger words. This strong performance highlights a key advantage of WARP’s design: by intervening at the retrieval layer rather than the model layer, it transfers robustly across architectures and maintains effectiveness even in substantially different model environments.